REPORT

Title

Implementation of the Eight Puzzle Problem Using Python

Aim

To implement the Eight Puzzle Problem in Python and find a sequence of valid moves from the initial state to the goal state.

Problem Statement

The Eight Puzzle Problem is a sliding tile puzzle consisting of eight numbered tiles and one blank space arranged in a 3 × 3 grid. The objective is to transform the given initial arrangement into a desired goal arrangement by sliding one adjacent tile into the blank space at a time. The program should determine a valid sequence of moves that reaches the goal configuration while following the puzzle constraints.

Requirements

- Python 3.x

Procedure

1. Represent the puzzle using a 3 × 3 matrix.
2. Define the initial and goal configurations.
3. Locate the blank tile in the current state.
4. Generate all valid moves by moving the blank tile.
5. Create new puzzle states after each valid move.
6. Keep track of visited states to avoid repetition.
7. Continue exploring states until the goal configuration is found.
8. Display the sequence of puzzle states from the initial state to the goal state.

Program

```
from collections import deque


def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    moves = {
        0: [1, 3], 1: [0, 2, 4], 2: [1, 5],
        3: [0, 4, 6], 4: [1, 3, 5, 7], 5: [2, 4, 8],
        6: [3, 7], 7: [4, 6, 8], 8: [5, 7]
    }

    for move in moves[zero]:
        new_state = list(state)
        new_state[zero], new_state[move] = new_state[move], new_state[zero]
        neighbors.append(tuple(new_state))

    return neighbors


def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()
```

```
    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)

        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))

    return None

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
```

```
        7, 8, 0)

solution = bfs(start, goal)

if solution:
    print("Solution Found:\n")
    for step in solution:
        print(step[0:3])
        print(step[3:6])
        print(step[6:9])
        print()
else:
    print("No Solution Exists")
```

Output

```
Solution Found:

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Result

The Eight Puzzle Problem was successfully implemented in Python. The program generated valid puzzle states and found a sequence of moves that transformed the initial configuration into the specified goal configuration.

Conclusion

The Eight Puzzle Problem demonstrates state-space search and problem-solving techniques in Artificial Intelligence. It is commonly used to evaluate search algorithms such as Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search, Greedy Best-First Search, and A* Search. The problem illustrates how systematic exploration of states can be used to reach an optimal or feasible solution while avoiding repeated states.